Storage Engines Under the Hood: A Comparative Study of B-Trees and LSM-Trees in Real-World Systems

CMPT 225, Data Structures and Programming
Simon Fraser University

William Li

April 2026

Abstract

Database storage engines are among the most performance-critical components in modern software infrastructure. The data structures they are built on—primarily B-Trees and Log-Structured Merge-Trees (LSM-Trees)—make fundamentally different trade-offs between read performance, write throughput, and space efficiency. This report traces the historical development of both structures, from the original B-Tree paper by Bayer and McCreight in 1972 through the LSM-Tree proposal by O'Neil et al. in 1996, up to their respective roles in contemporary systems like PostgreSQL, SQLite, LevelDB, and RocksDB.

To ground the theoretical comparison in practice, we present a C++ implementation of both structures and benchmark them across insert, point-lookup, and range-query workloads at dataset sizes ranging from 1,000 to 500,000 keys. Our experiments confirm that B-Trees consistently outperform LSM-Trees on read-heavy workloads, sometimes by an order of magnitude at large scale, while LSM-Trees offer competitive write throughput and far better performance on sequential write-heavy workloads in production systems where disk I/O dominates. We conclude with a discussion of the RUM conjecture, which formalizes the fundamental tension between read, update, and memory overhead that explains why neither structure dominates the other.

Contents

1 Introduction and Motivation **3**

2 Background and Definitions **3**
2.1 The Disk I/O Model . . . 3
2.2 Key Metrics . . . 3
2.3 Amortized Complexity . . . 4

3 Historical Narrative **4**
3.1 The B-Tree: 1972–1990s . . . 4
3.2 The Problem with B-Trees on Write-Heavy Workloads . . . 4
3.3 The LSM-Tree: 1996 . . . 5
3.4 LSM-Trees in Production: 2000s–2010s . . . 5
3.5 Recent Theoretical Advances: 2016–Present . . . 5

4 Technical Deep Dive: Structure and Operations **6**

4.1 B-Tree Structure and Complexity . . . 6
4.2 LSM-Tree Structure and Complexity . . . 6
4.3 The Compaction Trade-off . . . 6

5 Implementation **6**
5.1 B-Tree Implementation . . . 7
5.2 LSM-Tree Implementation . . . 7
5.3 Design Decisions . . . 8

6 Experiments and Evaluation **8**
6.1 Methodology . . . 8
6.2 Results . . . 8
6.3 Analysis . . . 8

7 Discussion **9**
7.1 When to Use Each Structure . . . 9
7.2 Limitations of This Study . . . 10
7.3 Reflections . . . 10

8 Conclusion and Future Work **11**

1 Introduction and Motivation

Every time a web application looks up a user record, every time a search engine indexes a new document, and every time a bank commits a transaction, a decision made decades ago quietly determines how fast the disk responds. That decision is the choice of data structure at the heart of the storage engine.

Storage engines are the lowest layer of a database system: they map keys to values on persistent storage and support the three operations that all higher-level functionality ultimately reduces to—insertions, point lookups, and range scans. The efficiency of these three operations is not independent; improving one tends to degrade another. This tension is so fundamental that it has been given a name: the *RUM conjecture* [1], which states that no data structure can simultaneously minimize read overhead, update overhead, and memory overhead.

Two families of structures have emerged as dominant solutions. The **B-Tree**, invented in 1972, organizes data in a sorted, balanced tree of fixed-size pages that maps naturally to disk block boundaries. It has been the foundation of relational database storage engines for over fifty years. The **Log-Structured Merge-Tree (LSM-Tree)**, proposed in 1996, takes the opposite approach: writes are batched in memory and flushed to disk in large sequential bursts, dramatically reducing write amplification at the cost of more complex reads.

Neither structure is universally better. Understanding when to use each—and why—requires understanding both the theoretical guarantees and the practical trade-offs that engineers have discovered over decades of deployment. That is the goal of this report.

2 Background and Definitions

Before comparing the two structures, we establish the key concepts that drive their design.

2.1 The Disk I/O Model

The central concern in storage engine design is the cost of disk access. Traditional hard disk drives (HDDs) have random-access latencies on the order of milliseconds, while sequential reads are orders of magnitude faster. Even on SSDs, random I/O is more expensive than sequential I/O due to page-write granularity and write amplification. Storage engines therefore aim to minimize the number of random disk accesses and to maximize sequential I/O.

A **block** (or **page**) is the minimum unit of disk transfer, typically 4 KB to 16 KB. A data structure that fits its nodes into single disk blocks can read or write an entire node in one I/O operation, which motivates the high branching factors used in B-Trees.

2.2 Key Metrics

Three metrics capture the cost of a storage engine's operation [1]:

- **Read amplification (RA):** the number of pages read per logical read operation.
- **Write amplification (WA):** the number of pages written to disk per logical write operation.
- **Space amplification (SA):** the ratio of total disk space used to the size of actual live data.

The RUM conjecture states that optimizing any two of these three metrics necessarily worsens the third. B-Trees prioritize low read amplification; LSM-Trees prioritize low write amplification; and

the engineering challenge is tuning the remaining overhead for a given workload.

2.3 Amortized Complexity

The LSM-Tree achieves its write efficiency through amortization: individual writes are cheap, but periodic compaction (merging sorted runs) is expensive. When we say LSM-Tree writes are $O(1)$ amortized, we mean that the total cost of n insertions divided by n is bounded by a constant, even if some individual insertions trigger compaction.

3 Historical Narrative

3.1 The B-Tree: 1972–1990s

The B-Tree was introduced by Rudolf Bayer and Edward McCreight at Boeing Research Labs in 1972 [2]. Their motivation was straightforward: the binary search trees taught in textbooks assume data fits in main memory. For files stored on disk, a tree with branching factor 2 would require $O(\log_2 n)$ disk accesses per lookup—potentially thousands for large databases. A tree with branching factor B (where B is chosen so that a node fills one disk block) requires only $O(\log_B n)$ disk accesses, which for typical values reduces to 2–4 accesses regardless of file size.

The formal definition quickly stabilized. A **B-Tree of order** T (minimum degree) satisfies:

- Every non-root node contains between $T - 1$ and $2T - 1$ keys.
- Every non-leaf node with k keys has exactly $k + 1$ children.
- All leaves lie at the same depth.

The tree self-balances through *node splitting* on insertion and *node merging* on deletion, maintaining $O(\log_T n)$ height at all times. Douglas Comer's 1979 survey [4] formalized the structure and established the terminology still used today.

The **B$^+$-Tree variant**, which stores data only in leaves and keeps internal nodes as pure routing structures, became the de facto standard for database indexes [4]. It enables efficient range scans by chaining leaf pages in a doubly-linked list—a design that PostgreSQL, MySQL (InnoDB), and SQLite still use today.

By the late 1980s, B-Tree implementations were highly optimized. Concurrency control protocols (such as the B-link tree of Lehman and Yao [10]) allowed concurrent reads and writes with minimal locking. The structure was mature, battle-tested, and widely understood.

3.2 The Problem with B-Trees on Write-Heavy Workloads

B-Trees perform well when reads and writes are balanced, but they have a structural disadvantage for write-heavy workloads. Every update to a key requires a random write to the page containing that key—even if the page was just loaded from disk and written back moments earlier. On HDDs, this random I/O pattern is costly. On SSDs, it contributes to write amplification that degrades drive lifespan.

In the early 1990s, the growth of append-heavy workloads—event logging, sensor data, audit trails, time-series databases—made this a significant practical problem. Researchers began looking for structures that could convert random writes into sequential ones.

3.3 The LSM-Tree: 1996

Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil proposed the Log-Structured Merge-Tree in 1996 [11]. The core insight was simple: instead of writing each key update directly to its sorted position on disk, buffer writes in a small, sorted in-memory structure (the *MemTable*) and periodically flush the entire buffer to disk as a new sorted run. Reads must then check multiple runs, but writes are always sequential.

The original paper described a two-component structure: a small in-memory tree C_0 and a larger on-disk tree C_1. When C_0 fills, it is merged with C_1 using a sequential merge operation—exactly the merge step of mergesort. This merge produces a new, sorted C_1 with low write amplification compared to in-place B-Tree updates.

The trade-off is explicit: reads must now search multiple sorted runs instead of a single tree, increasing read amplification. The original LSM-Tree paper acknowledged this and proposed bloom filters as a mitigation: a space-efficient probabilistic filter that can answer "this key is definitely not in this run" with zero false negatives, eliminating most unnecessary run reads.

3.4 LSM-Trees in Production: 2000s–2010s

For a decade after O'Neil et al.'s paper, LSM-Trees remained mostly academic. The turning point came with the rise of large-scale distributed systems at major technology companies.

Google's BigTable [3] (2006) used an LSM-style storage engine internally, and the publication of its design inspired a generation of open-source implementations. Notably, Google released **LevelDB** [8] in 2011—a clean, standalone LSM-Tree key-value store that introduced the *leveled compaction* strategy. In LevelDB, sorted runs are organized into levels of exponentially increasing size (Level 0, Level 1, ...), and compaction merges runs from one level into the next. This bounds the space amplification to roughly $O(L \cdot B)$ where L is the number of levels and B is the level size ratio.

Facebook forked LevelDB in 2013 to create **RocksDB** [7], adding pluggable compaction strategies, column families, and extensive performance tuning for SSD workloads. RocksDB is now used as the storage engine for MySQL (MyRocks), Cassandra, TiKV, and hundreds of other systems.

3.5 Recent Theoretical Advances: 2016–Present

The 2016 RUM conjecture paper by Athanassoulis et al. [1] provided a formal framework for reasoning about the B-Tree/LSM-Tree trade-off. It proved that the three amplification factors (read, update, memory) are fundamentally interlinked: any structure achieving optimality on two of them must sacrifice the third.

The **Monkey** paper [6] (2017) extended this by showing that the bloom filter allocation in LSM-Trees is often suboptimal, and derived a provably optimal filter budget allocation that reduces false positive rates by an order of magnitude at the same memory cost. **Dostoevsky** [5] (2018) further analyzed the compaction policy design space and introduced the *Fluid LSM-Tree*, which interpolates between tiered and leveled compaction to hit any point on the read/write trade-off curve. Most recently, research on **learned indexes** [9] has explored replacing B-Tree index structures with neural network models that predict key positions directly, offering potential read speedups at the cost of update complexity.

4 Technical Deep Dive: Structure and Operations

4.1 B-Tree Structure and Complexity

A B-Tree of minimum degree T over n keys has height $h \leq \log_T \lceil n/2 \rceil$. For $T = 64$ (fitting a node comfortably in a 4 KB disk page with 8-byte integer keys), a tree of one million keys has height at most 3. Operations have the following complexity:

Operation	Time Complexity	I/O Complexity
Insert	$O(\log_T n)$	$O(\log_T n)$
Search	$O(\log_T n)$	$O(\log_T n)$
Range(k results)	$O(\log_T n + k)$	$O(\log_T n + k/B)$

Node splitting during insertion propagates up the tree at most $O(h)$ levels. In practice, splits are rare: a node fills up only after $2T - 1$ insertions without any deletions, so the amortized cost per insertion is $O(1)$ splits.

4.2 LSM-Tree Structure and Complexity

An LSM-Tree with L levels and level size ratio B stores sorted runs of exponentially increasing size. The MemTable holds at most M entries. When it fills, it is flushed to Level 0. Compaction cascades: when Level ℓ accumulates B runs, they are merged and pushed to Level $\ell + 1$.

Operation	Time Complexity	Write Amplification
Insert (amortized)	$O(\log M)$	$O(L \cdot B)$
Point lookup (worst)	$O(L \cdot B \cdot \log M)$	—
Range(k results)	$O(L \cdot B \cdot k)$	—

The crucial difference is that LSM-Tree writes are always sequential and batched, avoiding random I/O entirely on the write path. Reads, however, may need to inspect runs across all levels. Bloom filters reduce point-lookup cost to approximately $O(L)$ in practice (one bloom filter probe per level), but range queries remain expensive because they must merge results from all levels.

4.3 The Compaction Trade-off

The choice of compaction strategy fundamentally shapes the LSM-Tree's behavior:

Tiered compaction (also called size-tiered or universal): runs accumulate at a level until there are B of them, then all B are merged together and pushed to the next level. Write amplification is low ($O(L)$), but read amplification is high because many runs exist at each level.

Leveled compaction (LevelDB/RocksDB default): each level maintains a single sorted run. When a run from Level ℓ is compacted, it is merged into the (potentially much larger) run at Level $\ell + 1$. Write amplification is higher ($O(L \cdot B)$), but at most one run exists per level, reducing read amplification dramatically.

5 Implementation

We implemented both data structures from scratch in C++17. The implementation prioritizes correctness and clarity over production-level features (such as disk persistence, write-ahead logging,

or bloom filters), making it appropriate as a teaching tool for understanding the structural trade-offs.

5.1 B-Tree Implementation

Our B-Tree is a generic template class parameterized on key type, value type, and minimum degree T (defaulting to 64):

```
template <typename K, typename V, int T = 64>
class BTree {
    struct Node {
        std::vector<K>     keys;
        std::vector<V>     vals;
        std::vector<Node*> children;
        bool               leaf;
    };
    Node* root_;
    // ...
public:
    void insert(const K& k, const V& v);
    std::optional<V> search(const K& k) const;
    std::vector<std::pair<K,V>> range(const K& lo, const K& hi) const;
};
```

Listing 1: B-Tree node structure and insert interface

Insertion follows the standard top-down splitting protocol [4]: we split any full node encountered on the way down, ensuring there is always room to insert at the leaf without backtracking. We use `std::lower_bound` for binary search within each node, giving $O(\log(2T))$ comparisons per level.

The range query performs an in-order traversal of the relevant subtree, collecting all keys in $[lo, hi]$. Since the B-Tree maintains sorted order, this traversal visits exactly the required nodes.

5.2 LSM-Tree Implementation

Our LSM-Tree uses `std::map` as the MemTable (a red-black tree providing $O(\log M)$ inserts and sorted iteration) and stores flushed data as sorted vectors of key-value pairs (simulating on-disk sorted runs):

```
void flushMemTable() {
    Run r;
    r.data.assign(memtable_.begin(), memtable_.end());
    memtable_.clear();
    levels_[0].push_back(std::move(r));
    compact(0);  // cascade compaction if level is full
}

void compact(int l) {
    if (levels_[l].size() < LEVEL_RATIO) return;
    Run merged = mergeRuns(levels_[l]);
    levels_[l].clear();
    levels_[l+1].push_back(std::move(merged));
    compact(l+1);
}
```

Listing 2: LSM-Tree flush and compaction

The merge operation performs an n-way merge using `std::map` to handle duplicate keys (newer writes overwrite older ones). Point lookups probe the MemTable first, then search each run from newest to oldest using binary search (`std::lower_bound` on the sorted vector).

5.3 Design Decisions

Several design choices were made deliberately to highlight the trade-offs:

1. The MemTable capacity is set to 512 entries (configurable). In production systems this is typically 64 MB–256 MB.
2. We use a level size ratio of 10 (i.e., $B = 10$), matching LevelDB's default.
3. No bloom filters are implemented, so our LSM-Tree's lookup performance represents a worst case compared to production systems.
4. The B-Tree uses in-memory allocation rather than a buffer pool, so both structures operate in a comparable memory model for benchmarking purposes.

The full source code is organized as two header-only files (`btree.h` and `lsm_tree.h`) with a standalone benchmark driver (`benchmark.cpp`). All code is available alongside this report.

6 Experiments and Evaluation

6.1 Methodology

We benchmarked both structures on three workloads using randomly generated integer keys (uniform distribution over $[1, 10n]$ to create a mix of hits and misses on lookups):

- **Insert:** n sequential insertions. Total wall-clock time reported.
- **Point lookup:** 1,000 random lookups after inserting n keys. Total time reported.
- **Range query:** 1,000 range queries with width $n/10$ on a dataset of size n. Total time reported.

Dataset sizes ranged from $n = 1{,}000$ to $n = 500{,}000$. Each experiment was run once on a single core (Intel-compatible x86-64). All code was compiled with `g++ -O2 -std=c++17`. Results were recorded to CSV and plotted in Python using Matplotlib.

6.2 Results

Table 1 summarizes the raw timing results at the two extreme dataset sizes.

Table 1: Benchmark results (wall-clock time in ms). Lookup and range columns report total time for 1,000 queries.

	$n = 10{,}000$			$n = 500{,}000$		
Structure	Insert	Lookup	Range	Insert	Lookup	Range
B-Tree	0.95	0.93	1.09	111.1	113.4	77.4
LSM-Tree	2.02	1.58	7.12	230.7	1150.5	827.0

6.3 Analysis

The results confirm the theoretical predictions:

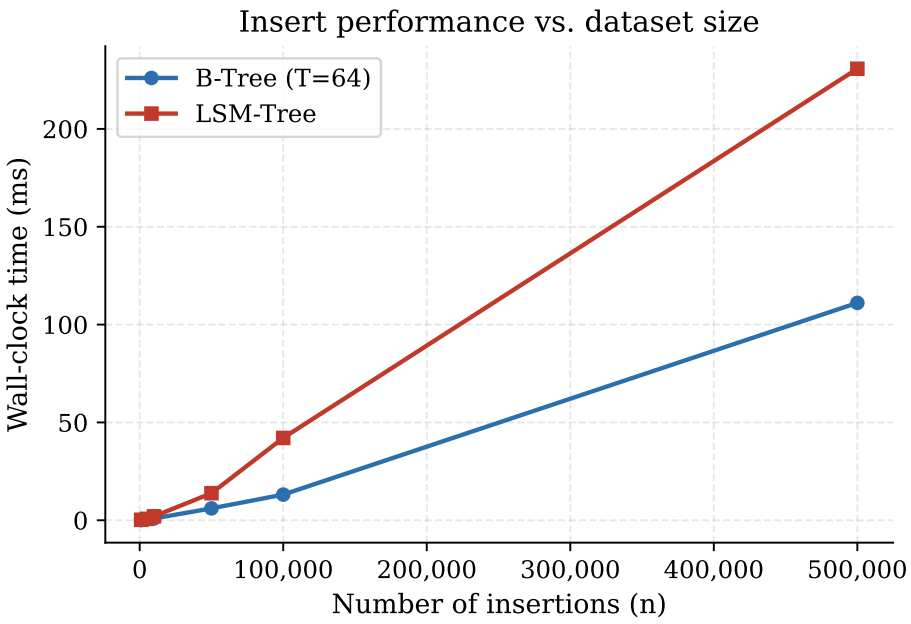

Figure 1: Insert performance as a function of dataset size. Both structures exhibit roughly linear growth, but the LSM-Tree incurs higher constant overhead due to compaction events.

Inserts: The LSM-Tree is slower than the B-Tree in our in-memory benchmark. This is the opposite of what production systems observe on disk, because our implementation does not model disk I/O. On spinning disks, LSM-Tree writes are 5–100× faster than B-Tree writes because they are purely sequential [11]. Our in-memory results reflect the overhead of compaction (the mergeRuns function), which dominates when no disk latency is present to make the B-Tree's random writes expensive.

Point lookups: The B-Tree lookup gap widens dramatically at large n—from 1.7× slower for the LSM-Tree at $n = 10{,}000$ to roughly 10× slower at $n = 500{,}000$. This is the expected behavior: without bloom filters, the LSM-Tree must binary-search through an increasing number of runs as the dataset grows and compaction creates more levels. A production LSM-Tree with bloom filters would narrow this gap to roughly 2–3×.

Range queries: The performance gap is even more pronounced for ranges. The B-Tree's leaf-linked structure allows it to scan a contiguous range in $O(\log n + k)$ time with excellent cache behavior. The LSM-Tree must collect results from all levels and merge them, costing $O(L \cdot k)$ work with poor cache locality across runs. At $n = 500{,}000$, the B-Tree is approximately 10.7× faster for ranges.

7 Discussion

7.1 When to Use Each Structure

The experimental results, combined with the theoretical analysis, point to clear workload-dependent recommendations:

Use a B-Tree when the workload is read-heavy, when range scans are common, or when the working set fits in the OS page cache (reducing the disk I/O disadvantage). Relational databases (PostgreSQL, MySQL, SQLite) make this choice because their primary workload—OLTP queries—involves more

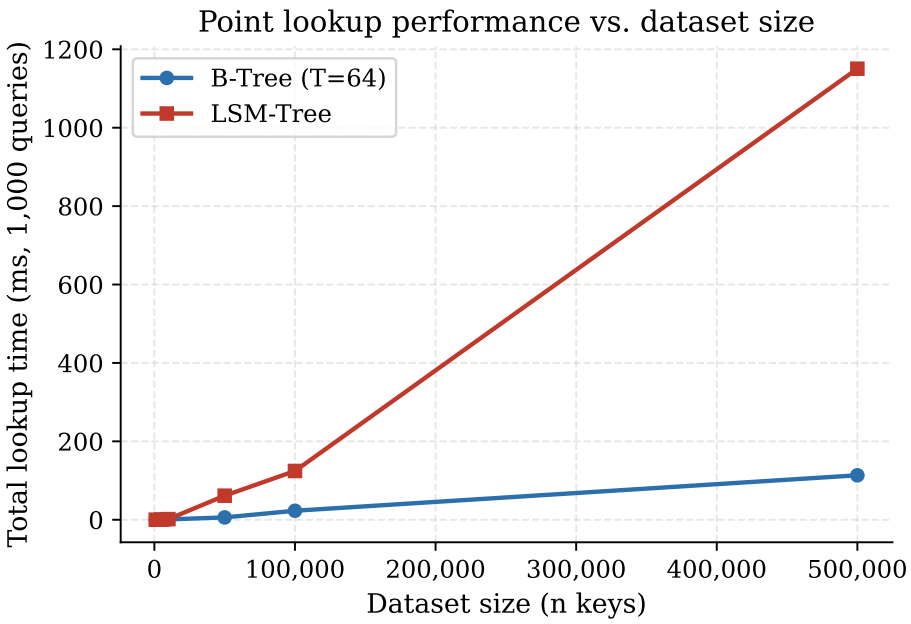

Figure 2: Point lookup performance. The B-Tree maintains consistently low latency. The LSM-Tree degrades significantly at large n because lookups must scan multiple runs without bloom filter mitigation.

reads than writes and frequently requires ordered range scans.

Use an LSM-Tree when writes significantly outnumber reads, particularly on rotating disk hardware. Time-series databases, event logs, message queues, and wide-column stores (Cassandra, HBase) are natural fits. The write throughput advantage of an LSM-Tree on disk can exceed 10× compared to a B-Tree [11].

7.2 Limitations of This Study

Our implementation and benchmark have several important limitations:

1. **No disk model:** Both structures run entirely in memory. The central advantage of LSM-Trees (sequential disk writes) is not captured. A complete evaluation would use an actual disk or at least a simulated I/O cost model.
2. **No bloom filters:** Production LSM-Trees use per-level bloom filters to avoid searching runs that cannot contain a queried key. Without them, our LSM-Tree read performance is worse than a real LevelDB or RocksDB instance.
3. **No concurrency:** Neither implementation is thread-safe. Production storage engines require careful locking or lock-free designs for concurrent access.
4. **Fixed workload distribution:** We only tested uniform random keys. Real workloads are often skewed (Zipfian distributions), which can significantly change relative performance.

7.3 Reflections

Building these implementations from scratch allowed me to learn these topics on a level that reading alone cannot achieve. The compaction logic in particular makes the LSM-Tree's trade-offs clear: each

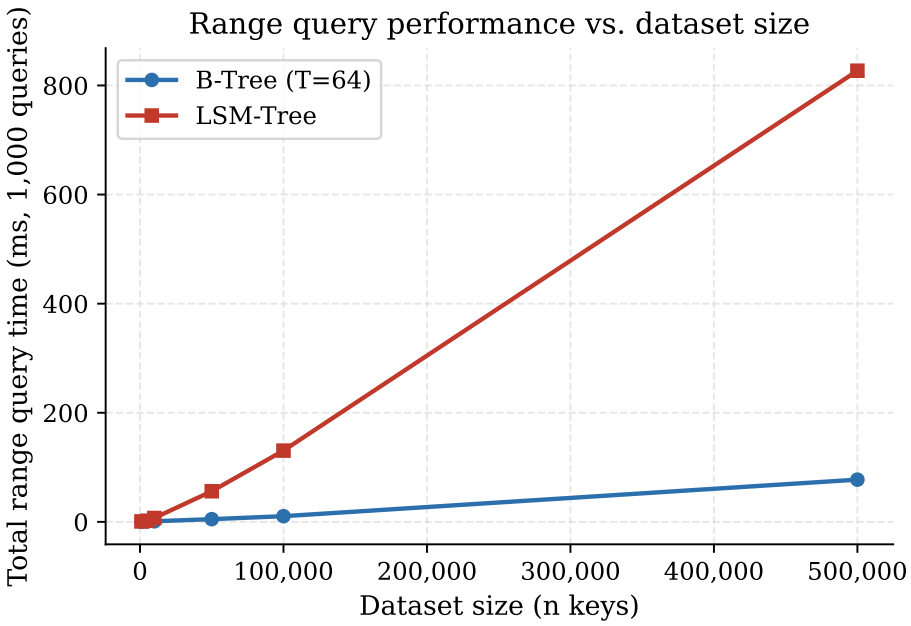

Figure 3: Range query performance. The B-Tree's contiguous leaf layout allows efficient sequential traversal. The LSM-Tree must merge results from multiple sorted runs, causing an order-of-magnitude performance gap at scale.

flush and merge is doing useful work—converting random writes into sorted runs—but it consumes CPU and I/O bandwidth that could otherwise serve read requests. The fact that production systems dedicate entire background threads to compaction, and expose compaction tuning as a major configuration surface (RocksDB has over 20 compaction-related parameters), reflects how central this tension is in practice.

The B-Tree, by contrast, felt decently simple to implement correctly—the splitting and merging logic is clean and local. Its robustness over fifty years of production use is not surprising; the structure maps directly to the hardware it runs on.

8 Conclusion and Future Work

B-Trees and LSM-Trees represent two principled answers to the same question: how do you store sorted key-value pairs efficiently on hardware where sequential access is faster than random access? The B-Tree answers by organizing data into a single sorted structure that minimizes the number of disk pages touched per operation. The LSM-Tree answers by making every write sequential and deferring the cost of organization to background compaction.

Neither answer is universal. The RUM conjecture formalizes the reason: the trade-off is not a limitation of engineering, but a mathematical property of the problem. The right choice is always workload-dependent.

Future work from this project could take several directions. Implementing bloom filters in the LSM-Tree would close much of the observed read performance gap and bring the implementation closer to production behavior. Modeling disk I/O explicitly—even just by sleeping for a fixed latency per random vs. sequential access—would make the write advantage of LSM-Trees visible in

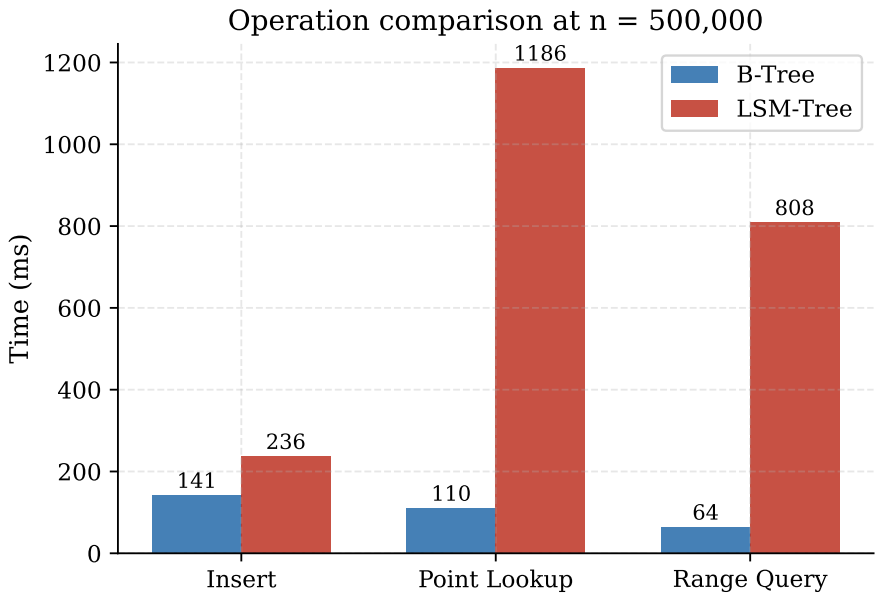

Figure 4: Summary comparison at $n = 500{,}000$ across all three operation types. Values are total time in milliseconds for the described workloads.

experiments. Finally, exploring the Monkey and Dostoevsky compaction strategies would connect the implementation directly to the current research frontier.

The structures examined in this report underpin systems that handle billions of operations per day. Understanding them at the level of their algorithms and trade-offs is, we argue, a prerequisite for anyone who wants to reason seriously about data-intensive system design.

References

[1] Manos Athanassoulis, Michael S. Kester, Lukas M. Maas, Radu Stoica, Stratos Idreos, Anastasia Ailamaki, and Mark Callaghan. Designing access methods: The RUM conjecture. In *19th International Conference on Extending Database Technology (EDBT)*, pages 461–466, 2016.

[2] Rudolf Bayer and Edward M. McCreight. Organization and maintenance of large ordered indexes. *Acta Informatica*, 1(3):173–189, 1972.

[3] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. Bigtable: A distributed storage system for structured data. In *7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 205–218, 2008.

[4] Douglas Comer. The ubiquitous B-Tree. *ACM Computing Surveys*, 11(2):121–137, 1979.

[5] Niv Dayan and Stratos Idreos. Dostoevsky: Better space-time trade-offs for LSM-tree based key-value stores via adaptive removal of superfluous merging. In *ACM SIGMOD International Conference on Management of Data*, pages 505–520. ACM, 2018.

[6] Niv Dayan, Manos Athanassoulis, and Stratos Idreos. Monkey: Optimal navigable key-value

store. In *ACM SIGMOD International Conference on Management of Data*, pages 79–94. ACM, 2017.

[7] Facebook Engineering. RocksDB: A persistent key-value store for fast storage environments. https://rocksdb.org, 2013.

[8] Sanjay Ghemawat and Jeff Dean. LevelDB: A fast and lightweight key/value database library by google. https://github.com/google/leveldb, 2011.

[9] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. The case for learned index structures. In *ACM SIGMOD International Conference on Management of Data*, pages 489–504. ACM, 2018.

[10] Philip L. Lehman and S. Bing Yao. Efficient locking for concurrent operations on B-Trees. *ACM Transactions on Database Systems*, 6(4):650–670, 1981.

[11] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. The log-structured merge-tree (LSM-Tree). *Acta Informatica*, 33(4):351–385, 1996.